Synchronization Latency for Databases: PowerSync versus ElectricSQL

Authors

Aníta Guðrún Finnsdóttir <agif@kth.se>

Olivia Hultman <oliviahu@kth.se>

Abstract

Sync engines are software component that supports the Seven Ideals for local-first software, which highlight the importance of collaboration between different clients. Previous work on this topic includes documentation reviews and developer surveys, leading us to the aim of this project: a quantitative experiment comparing the sync engines PowerSync and ElectricSQL. The experiment involves a setup including local and remote databases connected to the sync engines and repeated measurements of the synchronization latency, the so-called inconsistency window, under different network latency level settings. The timestamps are collected to create plots that visualize how the sync engines perform relative to each other, while also accounting for network latency. The median synchronization latency without added network latency was 17 and 47 ms for ElectricSQL and PowerSync, respectively. ElectricSQL synchronization latency was lower than PowerSync by a median of 26 to 30 ms across the tested network latencies. The increased latency when introducing network latency corresponded almost exactly to the added delay, with slopes of 1.02 and 1.05, respectively. This indicates that the difference between the engines could be more noticeable on fast networks. This research project can serve as a basis for the selection of the appropriate sync engine, since, to our knowledge, no empirical, independent, fair study of sync engines has been done before.

1 Introduction

1.1 Background and Rationale

An alternative to common cloud-based apps, in which data is stored on a remote server, is local-first software. In contrast to cloud-first software, this means the primary storage is hosted locally rather than relying on a local cache for only small parts of the data. A motivation for using the cloud is the collaborative opportunities it offers, but at the same time, it shifts control from the host to remote services. This means that the client depends on being connected to the internet and on abiding by the service's rules and status. By storing the data locally, the client is able to use and manipulate the data despite network and server failures. To combine the benefits of both local and remote storage, Kleppmann et al. [1] define the Seven Ideals that are desirable to achieve for local-first software. This includes short response times, synchronization between clients, offline work capabilities, conflict-free collaboration, long-lived data, inherent data security, and unrestricted control.

Especially for ideals 2-4, Kleppmann et al. [1] highlight the importance and need for synchronization among different clients to enable collaboration. One way to tick off these three ideals, and possibly a few more, is by using a sync engine. A sync engine has the responsibility to apply changes to all replicas of the data, solve conflicts between client updates, and make sure the storage system eventually converges after a network partition. Shiferaw [2] identifies three types of sync engines available, including relational partial-sync, where you have a local relational database that is partly synchronized with the backend database. Other types include document-based CRDTs and OT-based sync engines that are more focused on collaboration and conflict resolution.

1.2 Theory and Related Work

The existing comparative work on local-first sync engines is documentary and perceptual rather than measured. Shiferaw [2] compares six engines against the Seven Ideals using public sources, surveys 12 developers about their experiences, and explicitly identifies controlled laboratory benchmarking as the required next step. Performance figures for relational partial-sync engines are primarily found in vendor documentation.

Measuring the synchronization delay between a write and its visibility to the client is not new. Bermbach and Tai [3] benchmark eventual consistency from the client's perspective in distributed storage systems, where the time between a committed write and the point of its visibility to the reader is referred to as the inconsistency window. This measurement method does not appear to have been applied to local-first sync engines.

Both PowerSync and ElectricSQL fall into the relational partial-sync category described by Shiferaw [2]. They also share the same starting point on the server side: both read the Postgres logical replication stream to pick up changes from the backend database [4], [5]. They differ in how a change travels from the sync service to the application code on the client. ElectricSQL serves shapes, which represent the data to be synchronized, over an HTTP API. Synchronization of a shape occurs in two phases: an initial sync, followed by live mode. The shape logs are sent in the form requested by the client [6], what happens after that is up to the developer. ElectricSQL offers two options for materializing the shapes log: with streaming messages or applying it to a data structure or database [7]. PowerSync uses a different approach. On the server side the PowerSync service first writes the changes to a separate bucket storage database, from which they are then streamed to the clients [5]. On the client side, the client keeps a local SQLite database that is automatically kept in sync with the backend database, and all

client reads are performed locally in SQLite [8]. This difference matters for measurement. In PowerSync, the point at which a change becomes readable is determined by the engine, when the row is available in SQLite. In ElectricSQL, that point is dependent on what the developer has built on top of the shape log.

1.3 Research Problem, Aim, and Research Questions

Previous work shows that no independent, reproducible, like-for-like measurement of synchronization latency across these engines appears to be available. To address this research gap, this project is focused on experimental measurements to evaluate sync engines. The experiment is narrowed down to test two of the sync engines for relational structural data, namely PowerSync and ElectricSQL, to fit within the scope of the project. The objective is to measure the time it takes for a new write in the backend database to be reflected in the local database on another client, which we refer to as the synchronization latency. This is the same observed value that Bermbach and Tai [3] discuss and refer to as the inconsistency window. To evaluate how this synchronization time would change in a distributed setting that is dependent on an operational network, network latency was simulated and varied during the experiments. The research questions that should be answered are the following:

- How do PowerSync and ElectricSQL compare in terms of synchronization latency?
- How is the synchronization latency for PowerSync and ElectricSQL affected when the network latency is increased?

2 Research Method

2.1 Research Design and Methodological Rationale

This project uses a controlled quantitative experiment with two variables, network latency and the sync engine, comparing two options: PowerSync and ElectricSQL. The response variable is synchronization latency: the elapsed time from immediately before an insert in the backend Postgres database until that change becomes readable by application code on a separate client. The process was then repeated for both of the engines while adding artificial network latency on the downstream path between the sync service and the client. All other elements of the system are held constant across the two sync engines, so that observed differences in latency can be attributed to the engines themselves.

One alternative design that was rejected was a documentary or feature-based comparison. This would mean that official documentation, repositories, and engineering blogs are read and compared. This design had already been done by Shiferaw [2], who scored six engines against Kleppmann's Seven Ideals using public sources, and its design doesn't answer the research questions in this paper. A documentary comparison can only report what the documentation says the engines are built to do, and it cannot produce times in milliseconds. The times published come from the vendors themselves and are measured differently, so they cannot be compared directly and also have a risk of being unconsciously biased.

Due to the very limited time bound for the project, the experiment will focus on only two sync engines (PowerSync and ElectricSQL), one parameter (network latency), and one metric (synchronization latency). These sync engines were chosen since they are well-known within the field and since they both are used for similar goals. Network latency has a significant impact on the operation of a distributed service and motivates the use of sync engines that support local-first operation, and that's why this was considered an interesting parameter to vary across

the experiments. The synchronization latency was chosen as the metric since both engines use the Postgres logical replication stream as their source of change events [4], [5]. This means that the upstream part of the pipeline is architecturally comparable, which makes it more likely that any measured difference comes from the engines' own synchronization paths.

2.2 Data, Materials, and System Under Test

The server-side setup is a Postgres database and the two different sync engines PowerSync and ElectricSQL, each of them deployed using Docker containers. For each engine, a client was hosted locally as a Node.js process. PowerSync has built-in use of a local SQLite database, while ElectricSQL is more open regarding the choice of local storage solution. That made it possible to use a SQLite database even for the ElectricSQL engine, which makes the results less dependent on the database type. The clients were simple applications whose only purpose was to receive the new data record from the sync engine into their local database and log the associated timestamps. Together with the system setup, a separate orchestrator was run for the experiments, likewise using Node.js, which had the responsibility of sending basic queries to the Postgres database. A simple table which includes the columns `id`, `payload`, and `inserted_at` was created in both the local SQLite and Postgres databases, to be targeted for the orchestrator and the clients. For adding the network latency parameter into the experiments, a Toxyproxy Docker container was also introduced with the role of simulating different levels of downstream network latency by operating as a proxy between the sync engine and the client. The research plan stated that the command `tc netem` was going to be used for this purpose, but during the implementation it was found out that by utilizing Toxyproxy we avoided network traffic shaping with privileges at the OS level. It also made it easy to apply latency only to the specific connection.

Data was collected during the experiment, with the summarized measurements recorded in separate CSV files for each engine. Each measurement was stored as a single row containing the network latency level, iteration number, event ID, insertion timestamps, notification timestamp, receipt timestamp, and the resulting synchronization latency. Measurements were defined by two timestamps, t_0 and t_1, with the difference between them representing the synchronization latency. t_0 is recorded immediately before the row insert into Postgres, and t_1 is recorded when the new record can be read by the client. For PowerSync, this is when the data can be read from the local SQLite database [8]. For ElectricSQL, where materialization is left to the developer [7], t_1 was recorded at the read timestamp, which means after the shape was manually applied to a local store, that is, the local SQLite database. This ensures both engines were measured when the data is readable locally, rather than when the change first reaches the client. Both timestamps were taken in the same process on the same machine using the same monotonic clock. This minimizes clock synchronization error. One alternative for defining the synchronization latency that was considered in the previous research plan was recording the start timestamp (t_0) in Postgres when the row is inserted. It was subsequently deemed to be more suitable to set both of the timestamps from the client machine, to avoid misalignment between clocks. Although several considerations were taken into account, the two different sync engines with their respective clients could still not be assumed to be able to be compared on an equal basis. For this reason, an additional timestamp, `notified_at`, were included in the results to open up more possibilities for reaching conclusions related to the overhead of each engine. This timestamp represents the time when the client were first made aware of a new change, before starting to process it, since this processing is done differently for the two engines.

2.3 Work Procedure and Data Collection

The sync engines were deployed in a self-hosted setup so that we can control the whole pipeline, since both engines allow this. Table 1 shows the different components and versions used for the specific experiment setup.

Table 1: Environment setup used in the experiments.

Hardware/Software	Version
Operating system	Linux Mint 22.3
Processor	Intel Core Ultra 7 155H
Memory	30 GiB RAM
Docker Engine	29.2.0
Node.js	25.0.0
PostgreSQL	`postgres:16`
PowerSync Service	`journeyapps/powersync-service:latest`
ElectricSQL	`electricsql/electric:latest`
Toxiproxy	`ghcr.io/shopify/toxiproxy:2.9.0`

The synchronization latency experiments were initiated by starting the necessary Docker containers specified in Docker Compose files, one for each engine. The Postgres container was launched with the command `postgres -c wal_level=logical`, which determines how much Postgres will store in its Write-Ahead Log that is used to log changes in the database. Postgres was configured to support logical replication, which is needed for the sync engines. PowerSync required some manual configuration when starting the container, including creating a user and database for internal bucket storage use and a replication role in Postgres.

The clients for each engine also needed to be running; these were started with the `npm run start` command for each workspace. ElectricSQL is an Elixir application that connects to Postgres with a connection string, reads the logical replication stream, and serves shapes, which represent the data to be synchronized, over an HTTP API [4]. Since this client has no built-in local database, each message received in the subscription stream, if not a control event, was manually inserted into the SQLite database. The `notified_at` timestamp was set when a change event was triggered, and the `received_at` timestamp when the new row was returned by a `SELECT` query. When starting the PowerSync client, an environment variable `POWERSYNC_TOKEN` is required to be specified, which states the token to be used when connecting to the PowerSync service. This token was generated by the script `generate-dev-token.mjs` based on the variable `POWERSYNC_JWT_SECRET`, which was the same random generated secret that was passed on to the PowerSync container at start. PowerSync is available through client SDKs released under Apache 2.0 [9]. Through this SDK, the local database and the PowerSync connection were configured. When the `events` table was changed, a callback was triggered that queried the latest five inserted rows from the local database. At this time, the `notified_at` timestamp was set, and was then followed by an iteration through the fetched rows to find the ones that hadn't been seen before and log the corresponding `received_at` timestamp.

The orchestrator was then started to carry out and collect the test runs. For each engine, three sweeps were run with 100 iterations each and the network latency levels 0, 25, 50, and 100 ms. Multiple repetitions were executed for each combination of parameters to avoid temporary external impact on a single result. The command was the following:

```
npm run start --workspace @sync-experiment/orchestrator -- --engine
```

```
[engine] --networkLatenciesMs 0,25,50,100
```

For one of the sweeps, these network latency levels were also specified in the opposite order, to avoid a correlation between time effects and the current network latency as a result of the order. The main data collection used identical write sequences for every combination of engine and network latency, where one row is written at a time to the database. Before starting to collect the initial measurements, a couple of warm-up runs were executed under the same conditions as the measured runs. This was carried out to make sure the system had some time to start up and stabilize to avoid the inclusion of temporary variations in the final results. Each result, in the form of two timestamps representing the start and end of the synchronization together with its metadata, was then written to a CSV file.

2.4 Data Analysis and Quality Assurance

From running the tests, we obtained a collection of CSV files with measurements in the form of start and stop timestamps for the synchronization, with their corresponding metadata, including the type of sync engine and the current latency level. We used Python with the Pandas library to load the CSV files and validate them by checking if all expected columns are included. The data was then cleaned by checking for duplicate event IDs, non-positive latency values, as well as non-numeric values in quantitative columns. An engine column was added, and the three newest runs per engine were combined into one.

The mean, median, standard deviation, min, max, and outlier count were calculated, grouped by engine and network latency. We also used NumPy to calculate the 95th percentile.

The model that was identified was the following:

$$\texttt{latency_ms} = \texttt{intercept} + \texttt{engine_effect} + \texttt{error}$$

Here, `intercept` represents the baseline level, with `engine_effect` adding the difference that can be related to the choice of sync engine. The value `error` is the latency that the model can't explain, and that depends on external factors.

For additional statistical analysis, we used SciPy to compare the two engines with Welch's t-test, which does not assume that both groups have the same variance, and the Mann-Whitney U test, which does not assume normally distributed data. Both of these tests determine the difference between the groups with PowerSync as the baseline: the t-test by looking at the distribution within the groups and the U test by looking at the ranks of the values within the groups. We also used NumPy to calculate a bootstrap 95% confidence interval for the difference in mean latency, and computed the correlation between iteration and latency to check whether latency changed during the run.

For the second research question, we used SciPy to fit a linear regression model for each engine separately, with the added network latency as the independent variable and the synchronization latency as the dependent variable:

$$\texttt{latency_ms} = \texttt{intercept} + \texttt{slop} * \texttt{network_latency_ms} + \texttt{error}$$

Here, intercept represents the synchronization latency without any added delay, and slope represents how many milliseconds of synchronization latency are added for each millisecond of network latency. A slope of 1 means that the delay is passed straight through. The slopes are reported with a 95% confidence interval and R^2, and were compared with a z-test. The median latency per level was plotted with a shaded interquartile range.

Since measurements within a run may not be independent, we also fitted one slope per run to check that the results were consistent across runs, and one run used the latency levels in reverse order to rule out effects of time. Outliers were counted with the 1.5×IQR rule but not removed, since they represent real synchronizations. Each latency was also split into delivery (until the client is notified) and local processing (until the row is readable).

Finally, we used Matplotlib to create the plots. For the first research question, we created a boxplot and a histogram to visualize the distribution of the latency measurements for each engine, and a plot showing latency over iterations. The main graph for the second research question had the added network delay in milliseconds on the x-axis (0, 25, 50, 100 ms) and measured synchronization latency on the y-axis. There was one line per engine drawn through the medians, with a shaded band indicating the interquartile range. This graph showed whether the two engines degraded in the same way. If both lines rose with roughly the same slope, then the network affected them equally. If one line rose faster than the other, then the engines responded differently to a worse network.

2.5 Use of AI Tools

We used AI to gain an overview of a concept or topic, find relevant sources, and help improve the natural and programming language. Grammarly was used during the writing process, while different AI chatbots were used for topic or concept outlines, literature identification, and implementation. Even though we used some AI assistance for this project, nothing from it was taken at face value; everything that was used in this research was reviewed and verified against sources. We used only our own words in the report, and AI chatbot responses were used only as a basis for understanding the ideas we discussed.

3 Results

3.1 Engine comparison at baseline

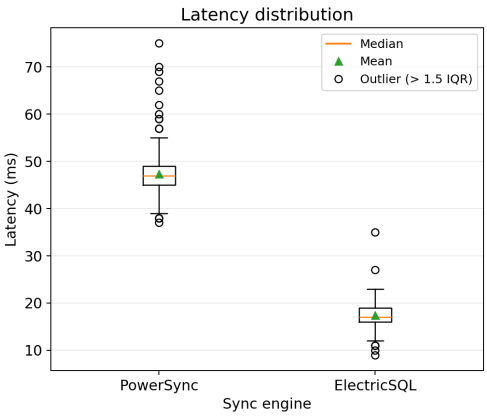
Table 2 shows the summary statistics for both engines. PowerSync had a mean latency of 47.38 ms and ElectricSQL 17.49 ms, which gives a difference of 29.89 ms with a 95% confidence interval of 29.327 to 30.473 ms. Welch's t-test showed that the difference in mean latency was statistically significant, with $t = 101.928$ and $p < 0.001$. The Mann-Whitney U test gave the same conclusions, with $U = 90000$ and $p < 0.001$. The U value is the highest possible for two groups of 300 measurements, which means that every PowerSync measurement was higher than every ElectricSQL measurement.

Table 2: Summary statistics of latency per engine.

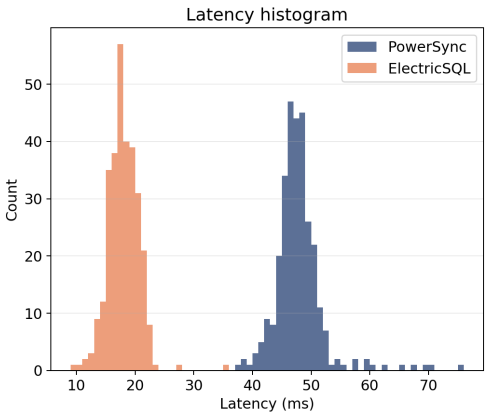
Engine	Mean	Median	Std. dev.	Min	Max	p95	Outliers
PowerSync	47.38 ms	47 ms	4.34 ms	37 ms	75 ms	52.05 ms	14
ElectricSQL	17.49 ms	17 ms	2.64 ms	9 ms	35 ms	21 ms	6

Figure 1 shows the distribution of the measurements for the engines. In both of the plots, the two engines do not overlap at all, as the slowest ElectricSQL measurement is still faster than the fastest PowerSync measurement. The histogram 1b shows a clear peak for each engine. ElectricSQL is more symmetric and has fewer outliers than PowerSync. PowerSync is mostly symmetric but slightly right-skewed, with a tail toward higher values. The same can also be seen in the boxplot 1a, with PowerSync having more marked outliers.

The engines' latency stayed roughly the same across iterations, and the correlation between



(a) Boxplot of the latency values



(b) Histogram of the latency values

Figure 1: Synchronization latency per engine without added network latency (n = 300 per engine)

iteration and latency was close to zero for both: 0.11 for PowerSync and -0.01 for ElectricSQL (see Figure 2). Most of the PowerSync outliers appear after iteration 50, with the highest at 75 ms.

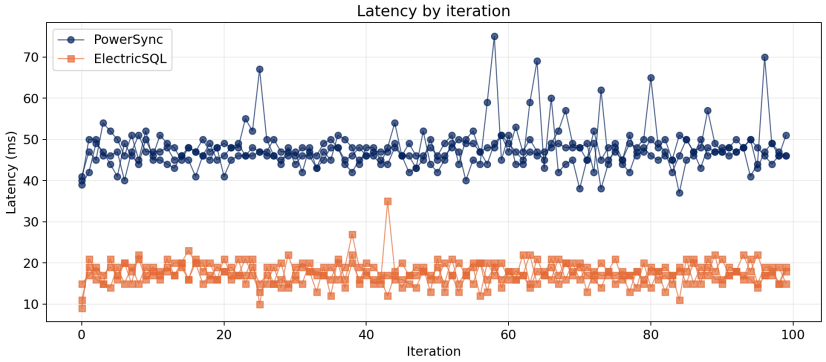


Figure 2: Synchronization latency per iteration without added network latency, with the three runs per engine shown together

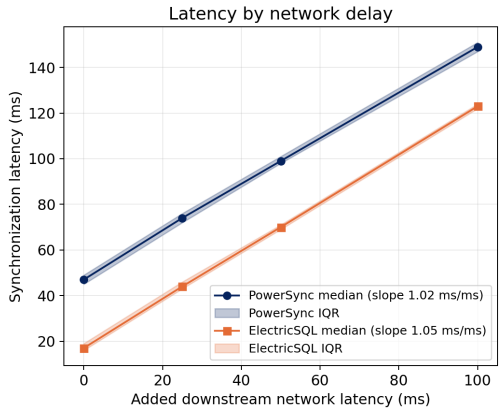
3.2 Effect of increased network latency

Table 3 shows the summary statistics with network latency for both engines. Synchronization latency rises with added delays for both engines: for PowerSynce, the median increased from 47 to 149 ms (0 to 100 ms), and for ElectricSQL, from 17 to 123 ms. ElectricSQL was faster at every level, but the median difference decreased from 30 ms at 0 ms to 26 ms at 100 ms. The spread stayed small at all levels, with a standard deviation between 2.2 and 5.5 ms. Although a few higher values occurred, such as 189 ms for PowerSync at 100 ms and 110 ms for ElectricSQL at 50 ms.

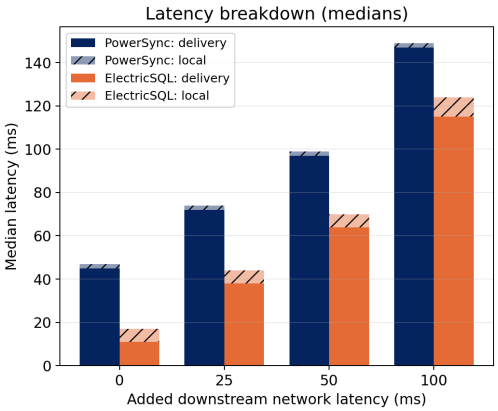
Table 3: Summary statistics of latency with network latency

Engin	Network latency	Mean	Median	Std. dev.	Min	Max	p95	Outliers
PowerSync	25 ms	74.62 ms	74 ms	5.49 ms	66 ms	113 ms	79 ms	8
PowerSync	50 ms	99.33 ms	99 ms	2.85 ms	90 ms	111 ms	104 ms	13
PowerSync	100 ms	149.92 ms	149 ms	4.7 ms	141 ms	189 ms	157 ms	14
ElectricSQL	25 ms	44.43 ms	44 ms	2.48 ms	37 ms	58 ms	47 ms	7
ElectricSQL	50 ms	70.42 ms	70 ms	3.06 ms	64 ms	110 ms	73 ms	9
ElectricSQL	100 ms	122.93 ms	123 ms	2.19 ms	114 ms	137 ms	126 ms	12

Figure 3a shows the median latency with the interquartile range. The linear regression gave a slope of 1.021 ms/ms for PowerSync (95% CI 1.014 to 1.028, $R^2 = 0.986$) and 1.053 ms/ms for ElectricSQL (95% CI 1.049 to 1.057, $R^2 = 0.995$). The difference of 0.032 ms/ms was statistically significant (95% CI 0.024 to 0.040, z = 7.88, $p < 0.001$). The result was consistent across the runs: the slopes per run were between 1.013 and 1.033 for PowerSync and between 1.049 and 1.055 for ElectricSQL, so every ElectricSQL run had a steeper slope than every PowerSync run. The median latency at each level differed by at most 1.5 ms between the runs, including the run with reversed order.



(a) Synchronization latency by simulated network latency



(b) Median synchronization latency split into delivery and local part

Figure 3: Synchronization latency per engine under added downstream network latency (n = 300 per engine and level).

Figure 3b shows the median latency split into delivery and local part. For PowerSync, the local part stayed at 2 ms at all levels, and the increase came from the delivery part, which rose from 45 ms to 147 ms. For ElectricSQL, the delivery part rose from 11 ms to 115 ms, and the local part was 6 ms up to 50 ms and 9 ms at 100 ms.

4 Discussion

4.1 Interpretation and Answers to the Research Questions

The first research question examined how PowerSync and ElectricSQL compared in terms of synchronization latency. In this baseline setup, ElectricSQL was clearly faster than PowerSync. ElectricSQL had a median of 17 ms compared to 47 ms, and the distributions did not overlap. Additionally, PowerSync had a wider spread and more high values. This means users would not only wait longer on average but also experience more variation. The latency breakdown

shows that most of the difference lies in the delivery part. In contrast, the local part accounts for only about 2 ms for PowerSync and 6 ms for ElectricSQL. Therefore, the majority of the difference is caused by what happens before the client is notified. One possible explanation is the architectural difference described in Section 1.2. The PowerSync service processes changes into its own storage before streaming them, and the client SDK writes the row to SQLite before notifying the application [5], [8]. Since our experiment only measured the time before and after these steps, it cannot show which of them causes the difference. Therefore, this explanation remains a hypothesis.

In the second research question, we examine how synchronization latency is affected as network latency increases. Both engines experienced an increase in latency almost exactly equal to the added delay. The slopes were 1.02 for PowerSync and 1.05 for ElectricSQL. This means that neither engine hides the delay nor makes it noticeably worse. The slope difference was statistically significant but small, only adding about 3 ms per 100 ms of delay. For ElectricSQL, the local part increased from 6 ms to 9 ms at 100 ms, which could partly explain the delay. However, we were unable to determine the cause.

One implication of these results is that the difference between the engines will be most apparent on fast networks. At 0 ms, engine differences accounted for most of the total latency, while at 100 ms, network delay did. Compared to earlier work that relied on documentation and developer surveys [2], these results provide measured values under identical conditions. Furthermore, we show that the inconsistency window approach from Bermabach and Tai [3] can be applied to local-first engines.

4.2 Validity, Limitations, and Generalizability

To ensure the validity of this study, the report includes information on the specific versions of the included components as well as the hardware and operating system used as the testbed. Also, the configuration for each component is documented, including the network latency levels and the number of repetitions executed. The implementation of the clients, the test script, and the collected measurements can be found in an open GitHub repository: https://github.com/OliviaHultman/powersync-vs-electricsql. This openness contributes to independent, transparent evaluation results, unlike the numbers published by vendors.

4.3 Ethics, Societal Aspects, and Sustainability

One ethical issue related to this project is fairness towards the vendors, as these are commercial products. There are many differences in the architectures of PowerSync and ElectricSQL, for example, the possible local storage solutions. To minimize this issue, an identical test environment setup was used for both sync engines, as much as possible. To limit the impact of randomness, each test was repeated multiple times, and the results were averaged. Other factors that could affect the results include the exact definition of the synchronization time metric and the choice of self-hosting over the cloud. When drawing conclusions and applying this study in a broader context, we make it clear that the results are valid only under these specific configurations.

An ethical motivation for this research on sync engines, and for continued research into them, comes from Kleppmann's 7 ideals for local-first software. Specifically, ideals 6 and 7, which concern security, privacy, ownership, and control of data [1]. These are some of the main reasons for local-first software over cloud-first software. This study could help developers navigate the landscape of local-first sync engines.

A social sustainability aspect is how well these sync engines perform under poor or unreliable

network connectivity, where local-first software matters the most. Although most of the world's population is now online, connectivity quality varies. Consequently, digital inclusion depends on systems remaining usable for people regardless of network strength [10]. Because local-first applications store data on the device, users can continue working when the connection is slow or unavailable [1], which could make collaborative tools more accessible to users constrained by unstable infrastructure, for example when working remotely. However, this benefit depends on how well the sync engine handles a poor connection when changes need to be shared with other users. Our research examines this one aspect by introducing a network-latency variable. However, given the limited scope of this project, this experiment does not fully capture how sync engines behave under different network conditions.

An environmental concern to consider when evaluating local-first software is whether moving data control and ownership to users reduces the load on data centers or just adds replication overhead, since every device holds a partial replica of the data. The balance between these two effects will also depend on each sync engine's implementation. Evaluating the trade-off between traffic and load for data centers is important as digital infrastructure becomes increasingly data center-based. Although this isn't the aim of our research, it is an important sustainability issue to discuss.

With the economy in mind, this research can reduce wasted engineering effort. Changing the sync engine is costly once an application is built on it, so comparing engines before committing to one can save engineering effort. The comparison between PowerSync and ElectricSQL can serve as a basis for choosing a sync engine, even though it should be used with caution and with the specific configuration and platform in mind. Differences between the sync services and their vendors, including openness and licensing, affect the long-term costs and independence for the developer.

4.4 Implications and Future Work

Since this study examined only synchronization latency between the backend and a single client, a logical next step would be to introduce a second client. This would allow measurement of the time required for a write operation on client A to become visible on client B, first under baseline conditions and then with additional network latency introduced. In this setup, three timestamps could be recorded: when client A commits the write, when the backend receives the write, and when it becomes visible on client B. Future work could also account for additional network performance variables such as jitter and packet loss.

5 Conclusion

The aim of this research was to compare two sync engines, PowerSync and ElectricSQL, in terms of synchronization latency over different network latencies. This was done in a controlled quantitative experiment. The sync engines were deployed in a self-hosted setup, with a server-side for each engine deployed in a Docker container and a simple client. The experiment showed that, with no added network latency, PowerSync's median latency was 47 ms and ElectricSQL's was 17 ms. That is a 30 ms difference between the engines, where PowerSync is 2.8x slower than ElectricSQL. When network latency was added (25, 50, and 100 ms), the latency increased almost exactly equal to the added delay for both engines. The slope difference between the engines was small (0.032 ms/ms), corresponding to only about 3 ms more latency per 100 ms of added delay. The slopes were 1.021 ms/ms and 1.053 ms/ms for PowerSync and ElectricSQL, respectively. This showed that the difference between the engines is more noticeable on fast networks. This was a very small-scale experiment, with a simple model and local setup, so the findings may not generalize to more complex real-world environments or workloads.

References

[1] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, “Local-first software,” April 2019, last accessed 3 September 2026. [Online]. Available: https://www.inkandswitch.com/essay/local-first/

[2] E. Shiferaw, “Sync engine comparison for localfirst web applications an empirical study,” Master’s thesis, Tampere University, Faculty of Information Technology and Communication Sciences, May 2026. [Online]. Available: https://trepo.tuni.fi/bitstream/handle/10024/237992/ShiferawErmyas.pdf?sequence=2

[3] D. Bermbach and S. Tai, “Eventual consistency: How soon is eventual? an evaluation of amazon s3’s consistency behavior,” December 2011, last accessed 11 September 2026. [Online]. Available: https://www.semanticscholar.org/paper/Eventual-consistency:-How-soon-is-eventual-An-of-Bermbach-Tai/ea079e2b3363ab9a8bd84a985f64407be5cf6673

[4] Electric, “Postgres sync,” n.d., last accessed 3 September 2026. [Online]. Available: https://electric.ax/sync/postgres-sync

[5] PowerSync, “Powersync service,” n.d., last accessed 8 October 2026. [Online]. Available: https://docs.powersync.com/architecture/powersync-service

[6] Electric, “Http api,” n.d., last accessed 11 September 2026. [Online]. Available: https://electric.ax/docs/sync/api/http

[7] ——, “Client development,” n.d., last accessed 11 September 2026. [Online]. Available: https://electric.ax/docs/sync/guides/client-development

[8] PowerSync, “Client architecture,” n.d., last accessed 8 October 2026. [Online]. Available: https://docs.powersync.com/architecture/client-architecture

[9] ——, “Open-source source-availability,” n.d., last accessed 3 September 2026. [Online]. Available: https://powersync.com/open-source

[10] ITU, “Measuring digital development facts and figures,” 2025, last accessed 6 October 2026. [Online]. Available: https://www.itu.int/hub/publication/d-ind-ict_mdd-2025-3/